Code559

Front-End Developer Course

MODULE 2

CSS Foundations

Give your pages color, type and spacing, and learn exactly which rule wins.

Free lesson: 559solutions.com/code559/module-02-css-foundations/

In this packet

1. **The lesson**
2. **Project: One stylesheet for the whole site**
3. **Finishing work in Git**
4. **Troubleshooting: five common problems**
5. **Self-assessment**
6. **Answers**

Powered by 559 Solutions

About this module

What this module covers

You learn to link a stylesheet, write rules for color, type and spacing, predict which rule wins, and build a navigation bar with Flexbox. It saves you hours: when a rule "does nothing," the cause is almost always one of the three problems this module teaches on purpose.

Tips for the learner

Some prompts ask you to explain code to yourself, out loud. Others ask you to write something down. Each habit does a different job.

Recite to understand. When a prompt says "out loud," explain the code in your own words, as if you were teaching it. Saying it aloud makes a deeper impression than reading silently. It helps you picture how the pieces of code fit together. And it shows you right away which line you cannot explain yet. That line is the one to review.

Write it down to check yourself. When a prompt says "write down," it is usually a prediction, a choice, or a guess about a bug. A written answer can't change after you see the result, so the comparison is fair and you learn from the difference. Writing also keeps your attention on the lesson and helps you pick out the main points.

Keep one notebook for this course. Use it for your predictions, key terms, steps, mistakes, and questions to come back to. Date each entry and write the module number, so you can find it later. Before you start a new module, read your last few entries and recite one or two of them out loud.

The four badges

Every core concept moves through the same four steps, called IEIA. Each step is marked with its badge.

 - **Introduce** (a lightbulb: a new idea) - You meet the concept: its name, and why it matters to you, in a sentence or two.

 - **Explain** (a speech bubble: the idea in words) - How it works, one idea at a time, in plain words.

 - **Illustrate** (an eye: see it) - A small example you can look at: a code snippet, a real screenshot or a diagram. Its caption, which starts with this badge, tells you what to notice.

 - **Apply** (a keyboard: your turn) - A short task to do right now, before you read on. Often you type or predict something; sometimes you write it down, sketch it, or explain code to yourself out loud.

Rights and credits

© 2026 559 Solutions LLC. This packet is licensed to the purchaser for personal use. Do not share, resell or post it.

Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Contents

About this module 2

Part 1: The lesson 4

- Why this matters 4
- Before you start 4
- Core concepts 5
  - 1. How does a page get its styles? 5
  - 2. What are the parts of a CSS rule? 6
  - 3. Which properties give you color, type and spacing? 6
  - 4. What is the difference between an element selector and a class selector? 7
  - 5. When two rules disagree, which one wins? 8
  - 6. What is the difference between block and inline boxes? 10
  - 7. How does Flexbox lay out a navigation bar? 10
- Exercise 12
- Common misconception 14
- Summary 14
- Check your understanding 14
- Before next module 15
- Go further 15
- What to keep 15

Part 2: Project: One stylesheet for the whole site 16

Part 3: Finishing work in Git 22

Part 4: Troubleshooting: five common problems 24

Part 5: Self-assessment 26

Part 6: Answers 27

About 559 Solutions 30

© 2026 559 Solutions LLC

PART 1

The lesson

Give your pages color, type and spacing, and learn exactly which rule wins.

Time: about 2.5 hours (estimate)

In this module you will learn:

- How to link an external stylesheet with `<link rel="stylesheet">`
- The parts of a CSS rule: selector, property, value, declaration
- Colors, typography and spacing: `color`, `background-color`, `font-family`, `font-size`, `line-height`, `margin`, `padding`
- Element selectors vs class selectors, writing reusable rules, and the descendant selector (`nav a`)
- The cascade and specificity: which rule wins when two rules disagree
- Block boxes vs inline boxes in normal flow
- Flexbox for a navigation bar: `display: flex`, `gap`, `justify-content`, `align-items`
- Saving packet-project work in `my-site` with the Git routine from Module 1

Why this matters

In Module 1 you wrote structure. Every page looked the same: black serif text on white. CSS is the layer that decides how things look. It is also the layer where beginners lose the most time, because a rule they wrote "does nothing" and they cannot see why.

Almost every time, the reason is one of three things: the stylesheet is not linked, another rule is winning, or the element is a different kind of box than they thought. This module teaches all three on purpose, with pages rendered in a real browser so you can see each rule win or lose.

By the end you will style a page with one shared stylesheet and build a navigation bar with Flexbox.

Before you start

1. Finish Module 1. Your `my-site` folder should be live on GitHub Pages, with your first page (or with your Home and About pages, if you did the Module 1 packet project).
2. Make a throwaway folder next to `my-site` called `practice-02`. Copy your `practice-01/index.html` and your image into it.
3. Open `practice-02` in VS Code (**File → Open Folder...**). Open `index.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window). After each save, press **F5** to reload. Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab.

© 2026 559 Solutions LLC

> **Checkpoint.** You can edit `practice-02/index.html`, save, refresh, and see the change. Your live site from Module 1 still opens.

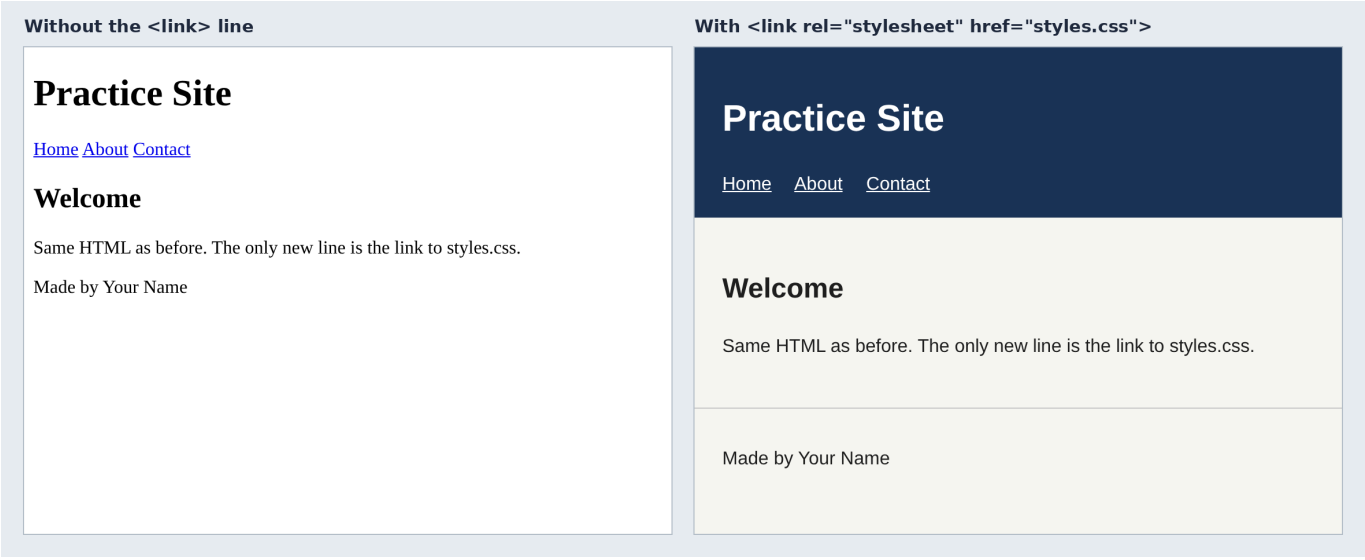
Core concepts

1. How does a page get its styles?

CSS lives in its own file, usually called `styles.css`, next to your HTML. A `<link>` element in the `<head>` tells the browser to fetch it.

`<link rel="stylesheet" href="styles.css">` has two parts. `rel="stylesheet"` says what kind of file it is. `href` is a relative path, exactly like the links you wrote in Module 1. The browser requests that file as a second request after the HTML. One stylesheet can be linked from every page of a site, so one change restyles all of them.

```
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Home | Practice Site</title>
  <link rel="stylesheet" href="styles.css">
</head>
```



The HTML is identical except for one `<link>` line. Each side was rendered at 560 px wide; the labels above them were added to the picture. (real: rendered in Chromium)

© 2026 559 Solutions LLC

Think this through: which part of the `<link>` element would you check first if a page showed no styles at all? (The `href`.)

2. What are the parts of a CSS rule?

A stylesheet is a list of **rules**. Each rule picks some elements and says how they should look.

The **selector** comes first and chooses the elements. Inside the curly braces are **declarations**. Each declaration is a **property**, a colon, a **value** and a semicolon. A missing colon breaks that one declaration. A missing semicolon breaks that declaration *and the next one*, because the browser reads the two as one long value that makes no sense. (The semicolon after the last declaration, just before `}`, is optional, so leaving out that one breaks nothing.) The browser shows no error, so look closely when a line "does nothing," and check the line above it too.



Selector, then declarations in braces. Colon between property and value, semicolon after.

Write a rule on paper that makes every `<h2>` green. Check it has a selector, braces, a colon and a semicolon.

3. Which properties give you color, type and spacing?

A handful of properties do most of the work on a simple page.

`color` sets text color; `background-color` sets the color behind it. Colors can be names (`white`) or hex codes (`#2b2d42`). `font-family` lists fonts in order of preference, ending with a generic family such as `serif` or `sans-serif` as a fallback. `font-size` sets text size, and `line-height` sets the space between lines (a plain number like `1.5` means 1.5 times the font size). `padding` is space *inside*

© 2026 559 Solutions LLC

an element's edge; `margin` is space *outside* it. You will study the box model properly in Module 3. Some properties are **inherited**: MDN notes that `color` is, so setting it on `body` colors the text inside it, while `margin`, `padding` and `border` are not.

```
body {
  font-family: Georgia, serif;
  color: #2b2d42;
  background-color: #fdf6ec;
  margin: 0;
  padding: 0 16px 24px;
}
```

The Exercise page later in this lesson uses exactly these properties.

Predict: if you set `color: purple` on `body`, what color will a paragraph be? (Purple, by inheritance, unless another rule sets the paragraph's color.)

4. What is the difference between an element selector and a class selector?

An **element selector** is a tag name, such as `p` or `h2`. A **class selector** starts with a dot, such as `.highlight`, and matches any element with `class="highlight"`.

Element selectors style every element of that kind, which is right for site-wide basics. Class selectors style only the elements you mark, which makes them **reusable**: write the rule once, then add the class to any element that needs it, on any page. Choose class names for what something *is* (`.highlight`, `.note`), not what it looks like today (`.orange`), so the name still makes sense after a redesign.

```
p { color: gray; }           /* every paragraph */
.highlight {                 /* only elements with class="highlight" */
  background-color: #edae49;
  padding: 8px 12px;
}
```

You can also join two selectors with a space. `nav a` is a **descendant selector**: it matches every `a` that sits anywhere inside a `nav`, and no other links. (MDN calls the space a *descendant combinator*.) You will use it in the Exercise to make only the nav links white.

```
nav a { color: white; }      /* links inside the nav; other links keep their color */
```

In the Exercise screenshot, only the paragraph with `class="highlight"` gets the orange background.

© 2026 559 Solutions LLC

Name one element on your Home page that should use an element selector and one that should use a class.

5. When two rules disagree, which one wins?

Often more than one rule matches the same element and sets the same property. The **cascade** is the browser's method for choosing one.

For the rules you write, two tests decide. First, **specificity**: MDN (*Handling conflicts*) describes an id selector as more specific than a class selector, and a class selector as more specific than an element selector. Compare the columns left to right: MDN says "a *million* **class** selectors combined would not be able to overwrite the specificity of *one* **id** selector." Second, only if specificity ties, **source order**: "the one that is defined last in the stylesheet is the one that will be used." So a later rule does not automatically win. It wins only against a rule of equal specificity.

Specificity: compare the columns from left to right

	ID	Class	Element	
`p`	0	0	1	weakest
`.note`	0	1	0	beats any element-only selector
`#special`	1	0	0	beats any number of classes
`.tag (twice)`	0	1	0	a tie: the later rule wins

Based on MDN, Handling conflicts. Only when every column ties does source order decide.

Read the columns from left to right; the first column that differs decides.

Here is a real test. To keep it apart from the Exercise, make two new files in `practice-02`: `cascade.html`, whose `<head>` links `cascade.css`, and `cascade.css` itself. The page's `<body>` holds four paragraphs. Each has an `id` so the Console picture below can find it; only `#special` has a rule.

```
<p id="one">1. Only the element rule (p) matches me.</p>
<p id="two" class="note">2. The element rule and the class rule (.note) both match me.
</p>
<p id="special" class="note">3. Element, class and id (#special) rules all match me.
</p>
<p id="four" class="tag">4. Two .tag rules match me. They tie, so the later one wins.
</p>
```

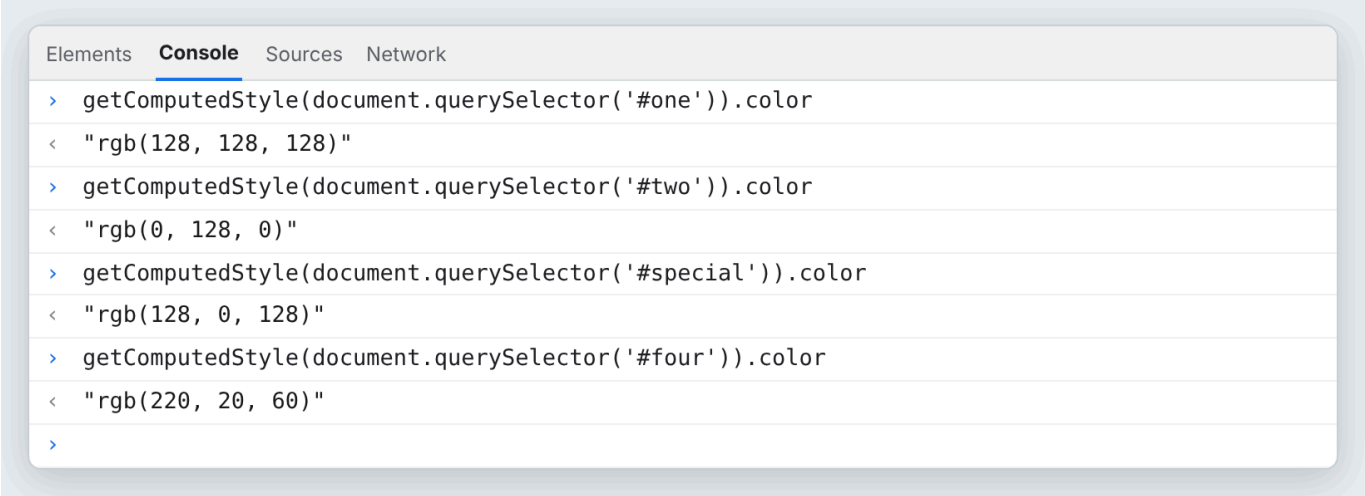

© 2026 559 Solutions LLC

The stylesheet sets `p` to gray, then `#special` to purple, then `.note` to green, then `.tag` to blue, and finally `.tag` to crimson. Notice that `#special` comes *before* `.note` in the file.

```
p        { color: gray; font-size: 20px; }
#special { color: purple; }
.note    { color: green; }
.tag     { color: blue; }
.tag     { color: crimson; }
```

1. Only the element rule (p) matches me.
2. The element rule and the class rule (.note) both match me.
3. Element, class and id (#special) rules all match me.
4. Two .tag rules match me. They tie, so the later one wins.

Line 3 is purple even though the `.note` rule comes later: the id is more specific. Line 4 is crimson because the two `.tag` rules tie and the later one wins. (real: rendered in Chromium)



The same page, asking the browser which color it actually used for each line: gray, green, purple, crimson. You do not need to type this yet; the Console is taught in the JavaScript modules. (real output, rendered)

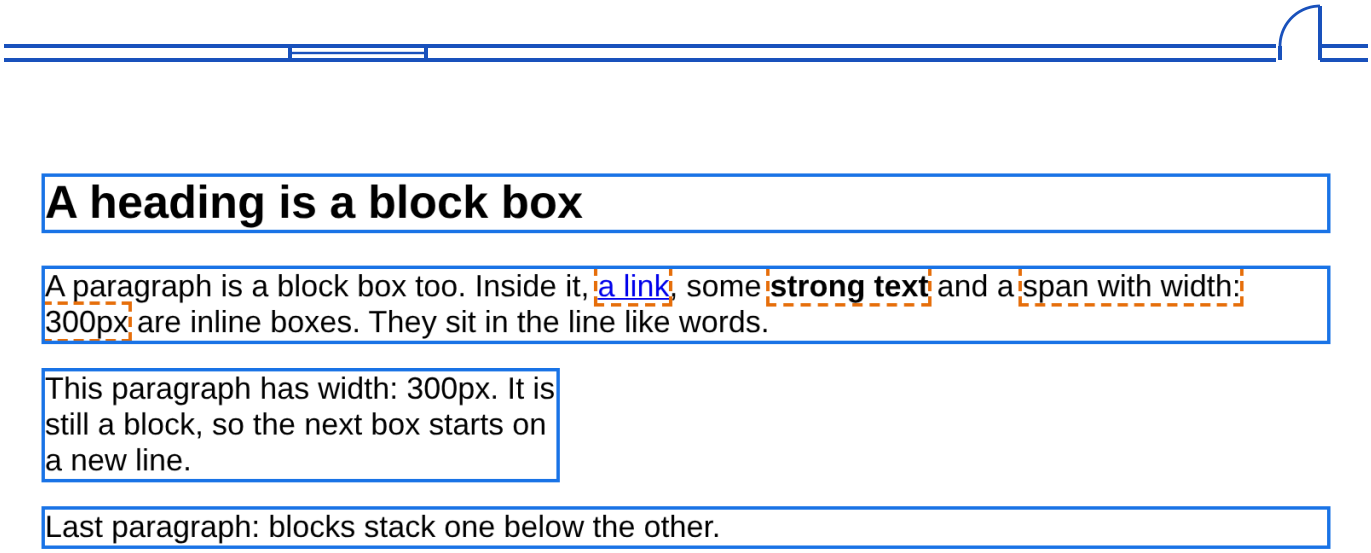
Predict: if you move the `#special` rule to the very end of the file, does line 3 change color? (No. It already had the highest specificity; order only breaks ties.)

© 2026 559 Solutions LLC

6. What is the difference between block and inline boxes?

Every element is drawn as a box. In normal flow, most boxes are either **block** or **inline**.

Block boxes, such as `h1`–`h6`, `p`, `ul`, `section` and `nav`, start on a new line and, by default, stretch across the full width of their container. MDN (*Block and inline layout in normal flow*) says block elements in English "lay out vertically, one below the other," and even with a width set "they will continue to lay out one below the other." Inline boxes, such as `a`, `span`, `em` and `strong`, sit in the line of text like words. MDN (*The box model*) says that for an inline box, "The `width`, `height`, and top and bottom margins will have no effect." When a `width` rule seems to do nothing, check whether the element is inline.



Blue = block, orange dashes = inline. The `span` has `width: 300px` and ignores it. The narrow paragraph obeys it, and the next block still starts on a new line. Measured at 800 px wide. (real: rendered in Chromium)

Add `outline: 2px solid red;` to a rule for `a` on any page and look at the shape of the boxes. Outlines are a quick way to see boxes without changing the layout.

7. How does Flexbox lay out a navigation bar?

Your nav links sit in a line because they are inline. To control spacing and alignment you turn their container into a **flex container** with `display: flex`.

`display: flex` goes on the *parent* (the `nav`). Its children become **flex items** and line up in a row. `gap` puts a fixed space between items. `justify-content` places items along the row: the default packs them at the start of the row; other values include `flex-end`, `center` or `space-between`, which pushes the first and last items to the edges and spreads the rest. `align-items` lines items up across

© 2026 559 Solutions LLC

the row; its default stretches them to the tallest item, and `center` centers them. One thing changes when you switch: the spaces from the line breaks between your `<a>` tags no longer show, so you use `gap` instead.

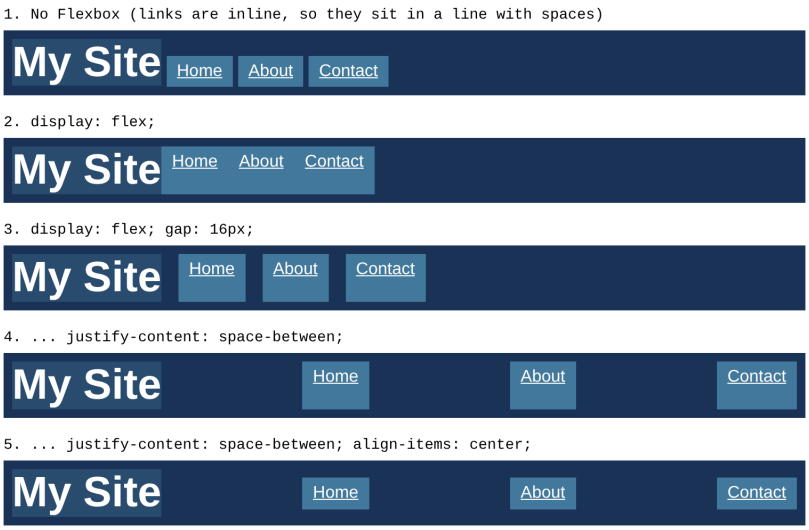
```
nav {
  display: flex;
  gap: 16px;
  justify-content: space-between;
  align-items: center;
}
```

The bars in the picture below hold a big **My Site** label and three links. In the picture the links also have a background color and padding, so you can see their boxes. To try it yourself, use this HTML with the `nav` rule above and add `.logo { font-size: 40px; }` (`href="#"` just goes to the top of the page):

```
<nav>
  <span class="logo">My Site</span>
  <a href="#">Home</a>
  <a href="#">About</a>
  <a href="#">Contact</a>
</nav>
```


© 2026 559 Solutions LLC

The big label is what makes `align-items` visible. When every item is the same height, like the two links in the Exercise nav, `align-items: center` changes nothing you can see.



Each bar adds one declaration. Compare 2 and 3 for `gap`, 3 and 4 for `justify-content`, and 4 and 5 for `align-items`: in 4 the links stretch to the height of the big label; in 5 they are centered. Measured at 800 px wide. (real: rendered in Chromium)

Predict what `justify-content: center` would do to bar 3. Then try it in the Exercise.

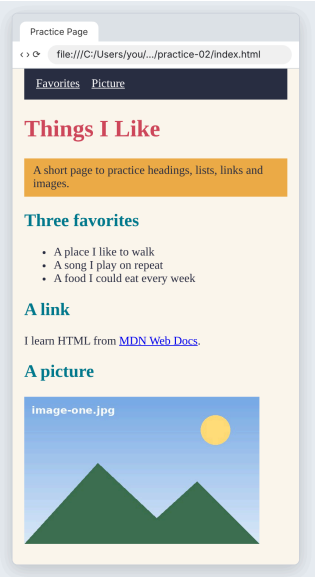
Exercise

Your hands, about 45 minutes (estimate). Work in `practice-02`. Type every line. Do not paste.

1. Create a new file in `practice-02` called `styles.css`.
2. In `index.html`, add `<link rel="stylesheet" href="styles.css">` as the last line inside `<head>`.
3. Add a `<nav>` at the top of `<body>` with two same-page links to two of your `<h2>` headings (give each heading an `id`, as in Module 1). Add `class="highlight"` to one paragraph.
4. **Predict.** Before you write any CSS, think this through: what will change when you add a `body` rule with a background color?
5. In `styles.css`, type a `body` rule (font, color, background, margin, padding), an `h1` rule and an `h2` rule with different colors, a `.highlight` rule, a `nav` rule with `display: flex`, `gap`, `padding` and a background color, and a `nav a` rule that makes the links white. Use your own colors.

© 2026 559 Solutions LLC

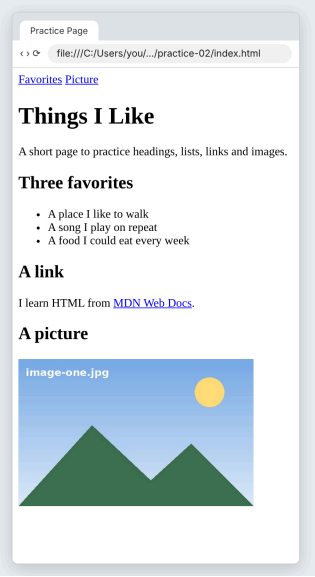
6. **Run.** Save both files and press **F5** in the browser.



One example, measured at 390 px wide, about the width of a phone. Your colors and words will differ. (page real: rendered in Chromium; browser frame drawn)

7. **Modify.** Add `justify-content: center;` to the `nav` rule. Save, refresh, and check your prediction from Concept 7. Then add a second `.highlight` rule *below* the first with a different background color. Which one wins, and why?

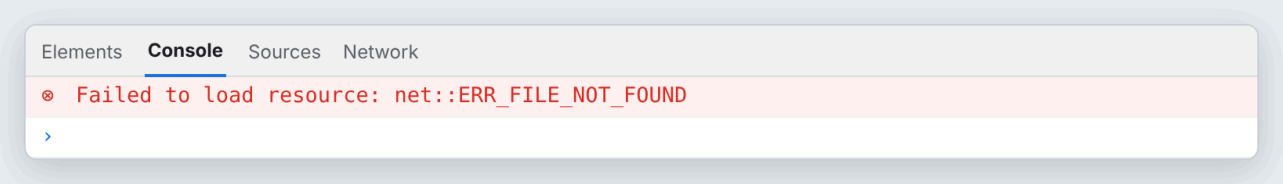
8. **Break it on purpose.** In `index.html`, change `href="styles.css"` to `href="style.css"` (one letter missing). Save and refresh.



Every style is gone, but nothing on the page says why. The HTML still works; only the request for the stylesheet failed. (page real: rendered in Chromium; browser frame drawn)

Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab.

© 2026 559 Solutions LLC



The browser did report the problem, just not on the page. In real DevTools the line also names the file it could not find. (real output, rendered; captured from Chromium loading the broken page from disk)

Edge and Chrome may lay out this message a little differently. Look for the red error and the file name it could not load. If the page came from a server instead of from disk (for example through Live Preview), the same mistake shows `Failed to load resource: the server responded with a status of 404 (Not Found)` instead: the server answered that it has no such file.

9. **Debug.** Fix the `href`, save and refresh. Then make a second, different mistake: delete the colon in one declaration, such as `color #d1495b;`. Which part of the page lost its style? Fix it.

Common misconception

> *"The rule lower down in the stylesheet always wins."*

What would you say? Hint: look again at line 3 in the cascade test, and at which rule came last.

Summary

A `<link rel="stylesheet">` in the `<head>` fetches a CSS file by a relative path, and a wrong `href` silently removes every style. A rule is a selector followed by declarations of property and value. `color`, `background-color`, `font-family`, `font-size`, `line-height`, `margin` and `padding` cover color, type and spacing, and some properties such as `color` are inherited. Element selectors style every element of a kind, while class selectors style only the elements you mark and can be reused anywhere; a selector with a space, such as `nav a`, matches elements inside other elements. When rules conflict, the more specific selector wins (id over class over element), and source order decides only a tie. Block boxes start on a new line and stack, while inline boxes sit in the line and ignore `width` and `height`. `display: flex` on a parent lines its children up in a row, and `gap`, `justify-content` and `align-items` control their spacing and alignment.

Check your understanding

1. Your page has no styles at all, but your CSS file is full of rules. What are the first two things you check?

© 2026 559 Solutions LLC

2. A `<p>` has `class="note"`. The stylesheet has `.note { color: green; }` and later `p { color: gray; }`. What color is the paragraph, and why?
3. **What happens if this changes?** A paragraph contains a link: `<p>Read <a href="https://developer.mozilla.org/">MDN</a> first.</p>`. You add the rule `p a { width: 200px; }`. What changes on the page, and why?

Before next module

- [ ] Your practice page links `styles.css` and shows your colors, fonts and spacing
- [ ] You broke the `href`, saw the unstyled page and the Console error, and fixed it
- [ ] You can explain why line 3 in the cascade test is purple
- [ ] Your nav uses `display: flex` with `gap`
- [ ] If you did this module's packet project in my-site, saved it with the routine: git status → git add . → git commit -m "..." → git push → check the live page. (Your practice folder is not saved to GitHub.)

Go further

- MDN: *Handling conflicts* (cascade, specificity and inheritance)
- MDN: *Block and inline layout in normal flow* and *The box model*
- MDN: search "Basic concepts of flexbox" and the reference pages for `justify-content` and `align-items`
- MDN: search "CSS selectors" for type and class selectors
- freeCodeCamp: the CSS section of the Responsive Web Design curriculum

---

What to keep

Keep your `practice-02` folder, including `cascade.html` and `cascade.css` if you typed the cascade test, and a one-line note in your own words of the rule for "which rule wins." Then answer for yourself: **which step would be hardest to repeat without this packet, and why?**

Code559 is a 559 Solutions project. Portions adapted from Visual Studio Code documentation (CC BY 3.0 US), GitHub Docs (CC BY 4.0), and GitHub Skills (MIT). Screenshots used with permission from Microsoft.

Sources checked 2026-10-09: MDN "Handling conflicts" (MDN's Learn page on cascade, specificity and inheritance); MDN "Block and inline layout in normal flow"; MDN "The box model". Checked 2026-10-11: MDN "Descendant combinator"; MDN "Introduction to CSS syntax: declarations, rulesets, and statements"; MDN "`<a>` HTML anchor element" (`href="#"` links to the top of the page); Live Preview source code (microsoft/vscode-livepreview on GitHub, `src/server/httpServer.ts`: a missing file gets status 404).

© 2026 559 Solutions LLC

PART 2

Project: One stylesheet for the whole site

Your hands, about 75 minutes (estimate). In the Exercise you styled a practice page in `practice-02`. Now you style your real site. You write one `styles.css` in `my-site` and link it from the Home and About pages you built in the Module 1 packet. One file sets the color, type and spacing for both pages, and one Flexbox rule turns the nav into a bar that looks the same on every page. You use only the CSS from Modules 1 and 2. Screen sizes and layout grids come in Module 3.

This project starts from the Module 1 packet's Home and About pages. If `my-site` still holds only your first practice page, build those two pages first (Module 1 packet, Part 2), then come back.

The brief

When you finish, `my-site` holds four files: `index.html` (Home), `about.html` (About), `image-one.jpg` and the new `styles.css`. Both pages link the same stylesheet with `<link rel="stylesheet" href="styles.css">` as the last line inside `<head>`. Nothing else in the `<head>` changes. The restyled site is online at `https://your-username.github.io/my-site/`.

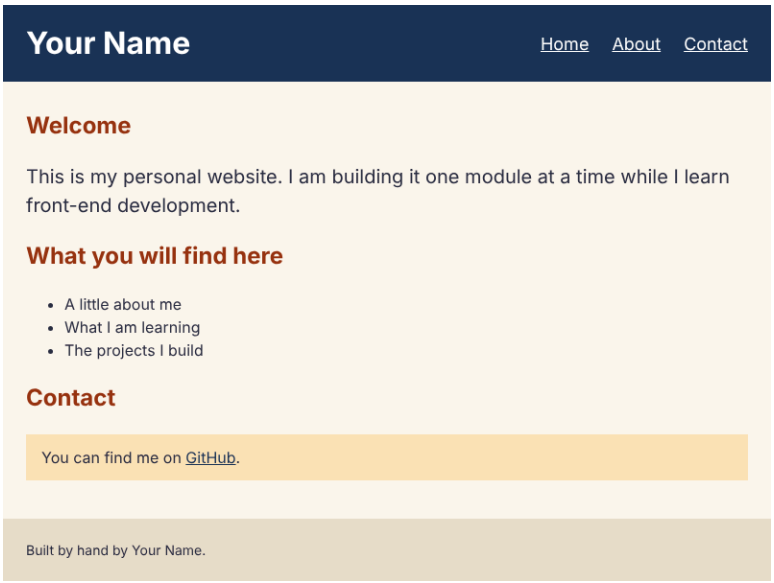
`styles.css` holds rules in four groups, in this order, each with a short comment above it:

- **Site-wide basics.** Element rules for `body` (font, text color, background color, line height, margin, padding), `h1`, `h2` and `a`. Because `color` is inherited, the `body` rule colors almost all the text on both pages.
- **Header and nav.** The `header` gets a dark background and light text. The `nav` gets `display: flex` and a `gap`, and a `nav a` rule makes only the nav links light. The header itself is also a flex container, so the site name sits on the left and the nav on the right.
- **Page areas.** `main` and `footer` get padding, and the footer gets its own background, so the page has three clear bands: header, main, footer.
- **Reusable classes.** At least two class rules, named for what the thing *is*. This project uses `.lead` (a larger opening paragraph) and `.note` (a paragraph in a colored box). Use `.note` on at least one paragraph on *each* page, so you can see one rule reused across pages.

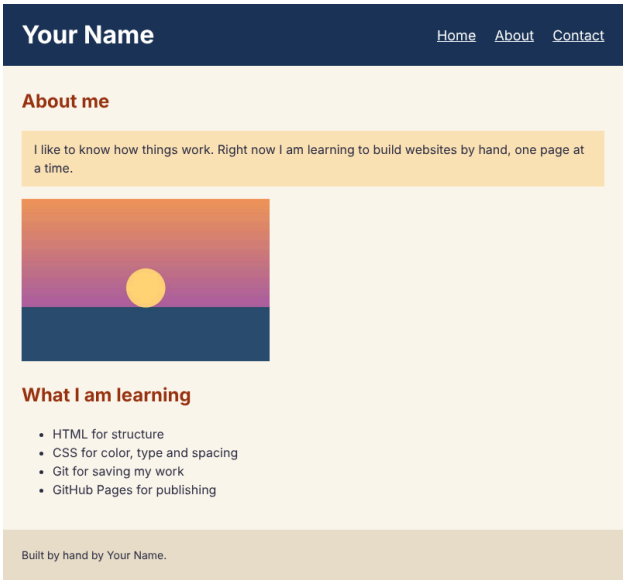
A visitor sees the same header, the same colors and the same fonts on both pages. Only the main content changes. The nav links sit in one row, in the same place, on both pages, and clicking between pages does not make the header jump. All the words, and all the colors, can be yours. The rule names above, the file name `styles.css` and the `<link>` line must match.

If you choose your own colors, keep body text at a contrast of at least 4.5:1 against its background. The sample colors below were checked: the dark text `#2b2d42` on the cream `#fdf6ec` is 12.6:1, white on the navy `#1d3557` is 12.4:1, the rust headings `#9a3412` on cream are 6.8:1, and the dark text on the `.note` box `#fde2b8` is 10.8:1 (computed with the WCAG contrast formula). Pale text on a pale background is the mistake to avoid. For links, `#1d3557` on the cream is 11.5:1; for the footer, `#e9dcc9` behind the dark text is 10.0:1.

© 2026 559 Solutions LLC



One example of the finished Home page. The nav is one row at the right of the header, the Contact paragraph uses `.note`, and the Welcome paragraph uses `.lead`. The sample asks for Segoe UI, a Windows font; this picture was made on a computer without it, so it shows a similar fallback font. Measured at 800 px wide. (real: rendered in Chromium at 800 px)



The About page with the same stylesheet. The header, footer and colors match the Home page, and the nav links sit at exactly the same spots (the browser measured both). The `.note` box is the same rule reused on a second page. (real: rendered in Chromium at 800 px)

The steps

Type every line. Do not paste. The `<link>` line is the same on both pages, but type it twice anyway: it is short, and typing it is how you learn to spot a wrong `href`.

© 2026 559 Solutions LLC

1. **Open `my-site`.** In VS Code, **File → Open Folder...** and choose `my-site` in your course folder, `C:\Users\you\Documents\code559`. The Explorer should show `about.html`, `image-one.jpg` and `index.html`. Open a terminal with **Terminal → New Terminal** and check that the prompt ends in `my-site>`. You will need it at the end.

2. **Plan on paper.** Choose four colors: a page background, a text color, a header background, and a heading color. Write their hex codes. Then choose a font list that ends in a generic family, for example `"Segoe UI", Arial, sans-serif`. If you are not sure about contrast, start with the sample colors from the brief and change them later.

3. **Predict.** Write down what you expect to see on the About page after you link the stylesheet from `index.html` only. You will check this in step 5.

4. **Create the stylesheet and link it from Home.** In the Explorer, create a new file in `my-site` called `styles.css`. Check the name in the Explorer: `styles`, with an `s`, then `.css`. Then open `index.html` and add the link as the last line inside `<head>`:

```
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Home - Your Name</title>
  <link rel="stylesheet" href="styles.css">
</head>
```

5. **Write the site-wide basics.** In `styles.css`, type the first group:

```
/* Site-wide basics */
body {
  font-family: "Segoe UI", Arial, sans-serif;
  color: #2b2d42;
  background-color: #fdf6ec;
  line-height: 1.5;
  margin: 0;
  padding: 0;
}

h1 {
  font-size: 32px;
  margin: 0;
}

/* Continue alone: an h2 rule with your heading color and a font-size,
   and an a rule that gives links a color that is easy to read on your
background. */
```


© 2026 559 Solutions LLC

A CSS comment starts with `/*` and ends with `*/`. Like an HTML comment, it never shows on the page. Keep the group comments; delete the "Continue alone" comment when you have typed your rules. Save both files. Open `index.html` in Edge or Chrome (double-click it in File Explorer, or drag it onto a browser window). After each save, press **F5** to reload. The Home page should show your font and colors. Now click **About** and compare with your prediction from step 3: the About page is still black serif text on white, because it does not link the stylesheet yet.

6. **Link it from About.** Open `about.html` and type the same `<link>` line as the last line inside its `<head>`. Save, go to the About page in the browser, and press F5. Both pages now share every rule you write. From here on, you edit one file and both pages change.

7. **Style the header and nav.** Add the second group:

```
/* Header and nav */
header {
  background-color: #1d3557;
  color: white;
  padding: 16px 24px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

nav {
  display: flex;
  gap: 24px;
}

/* Continue alone: a nav a rule that makes only the nav links white,
   with a font-size a little larger than the body text. */
```

   Read the `header` rule to yourself, line by line. It uses the same Flexbox ideas as the `nav`, one level up. The header's children are the `<h1>` and the `<nav>`, so they become two flex items. `justify-content: space-between` pushes the site name to the left edge and the nav to the right edge. `align-items: center` lines them up across the row, which matters here because the `<h1>` is taller than the links. The `nav` rule makes the three links the flex items of a second, smaller row, with `gap` between them.

   You need `nav a`, not plain `a`. A plain `a` rule set to white would also make the GitHub link in your Contact section white, on a cream background, where nobody could read it. The descendant selector picks only the links inside the `nav` (Concept 4).

8. **Run.** Save and press F5 on both pages. The header should be one dark band: the site name on the left, the three links in a row on the right. Switch between Home and About a few times and watch the header. It should not move at all.

© 2026 559 Solutions LLC

9. **Add the page areas.** Add the third group: a `main` rule with padding (for example `8px 24px 24px`), and a `footer` rule with its own background color, padding and a smaller `font-size`. Write this group alone. Save and refresh. The page now has three bands: header, main, footer. Because `body` has `margin: 0` and `padding: 0`, the header and footer bands reach the edges of the window.

10. **Write the reusable classes.** Add the fourth group:

    ```
    /* Reusable classes */
    .lead {
      font-size: 20px;
    }

    /* Continue alone: a .note rule with a light background color and padding. */
    ```

    Then use the classes in your HTML. On the Home page, add `class="lead"` to the Welcome paragraph and `class="note"` to the paragraph in the Contact section. On the About page, add `class="note"` to the first paragraph in About me. For example: `<p class="note">You can find me on …</p>`. Save all three files and refresh both pages. One rule now styles a paragraph on each page.

11. **Modify.** Change the header's `background-color` in `styles.css` to another dark color. Save and refresh *both* pages. Both changed, from one edit in one file. Put back the color you want.

12. **Which rule wins?** At the very end of `styles.css`, add this test rule:

    ```
    p { background-color: yellow; }
    ```

    Before you save, write down which paragraphs will turn yellow. Then save and refresh. Every paragraph turns yellow except the ones with `class="note"`. The `p` rule comes last, but the `.note` rule is more specific (class beats element), and source order decides only a tie (Concept 5). Your `.lead` paragraph does turn yellow: the `.lead` rule sets only `font-size`, so nothing disagrees with the `p` rule about the background. Delete the test rule, save, and refresh.

13. **Break it on purpose: flex on the wrong element.** In the `nav` rule, cut the line `display: flex;` and paste it into the `nav a` rule instead. Save and refresh. Predict first, then look.

© 2026 559 Solutions LLC



With `display: flex` on `nav a` instead of `nav`, the links stack in a column and the header grows taller. (real: rendered in Chromium at 800 px)

The links now stack, one under the other. Each `<a>` became a flex container of its own, which is a block-level box, so each one starts on a new line (Concept 6). The `nav` is a plain block again, so it has no flex items, and its `gap` does nothing. `display: flex` belongs on the *parent* whose children you want in a row (Concept 7). Move the line back into the `nav` rule, save, and refresh.

14. **Debug: a missing stylesheet on one page.** In `about.html` only, change `href="styles.css"` to `href="style.css"`. Save and refresh the About page. All of your styles are gone from that page, but the Home page is still styled. Press **F12** (or **Ctrl+Shift+J**) to open DevTools, then click the **Console** tab. You should see the red error `Failed to load resource: net::ERR_FILE_NOT_FOUND`, as in the lesson's Exercise. Which file is wrong: `styles.css`, or the page that links it? Fix the `href`, save, and refresh.

15. **Test both pages.** Open each page and tick the box.

    - [ ] Home and About both show your font, colors and three bands
    - [ ] On both pages, the nav is one row on the right of the header, in the same spot
    - [ ] On both pages, the nav links are light and every other link is easy to read
    - [ ] A `.note` box shows on both pages
    - [ ] All six nav clicks from Module 1 still work (three links on two pages)
    - [ ] The Console shows no red errors on either page

16. **Put it online.** Go to Part 3, *Finishing work in Git*, and publish your stylesheet with the save routine. Then run the test from step 15 again on the live site.

© 2026 559 Solutions LLC

PART 3

Finishing work in Git

Your styled pages work on your computer. Now you put them online with the save routine from Module 1: `git status`, `git add .`, `git commit`, `git push`, then check the live page. The routine is the same. What is new this time is a commit that adds a new kind of file and changes two old ones at once, and checking that the live site loads that file.

Items marked **NEW** are new in this part. The others repeat what you did in Module 1.

1. **Check the terminal.** In VS Code, with `my-site` open, look at the terminal prompt. It must end in `my-site>`. If it does not, use **File → Open Folder...**, choose `my-site`, and open a new terminal.

2. **See what changed.** Type:

   ```
   git status
   ```

   You should see `about.html` and `index.html` under **Changes not staged for commit**, each marked `modified:`, because you added the `<link>` line and the classes. You should see `styles.css` under **Untracked files**, because Git has never saved it. `image-one.jpg` is not listed. If you do not see `styles.css` at all, check its name in the Explorer (see *Troubleshooting*, problem 1).

3. **Stage the changes.** Type:

   ```
   git add .
   ```

   It prints nothing when it works. Run `git status` again. All three files are now under **Changes to be committed**: `modified: about.html`, `modified: index.html` and `new file: styles.css`.

4. **Commit (NEW: one commit for one change across several files).** Type:

   ```
   git commit -m "Style the site with one shared stylesheet"
   ```

   Git prints a short summary. The first line starts with `[main`, a seven-character commit id and your message. The next line says `3 files changed`, and a later line says `create mode 100644 styles.css`. Your commit id and numbers will differ. The new stylesheet and the two `<link>` lines belong together: one without the other is a broken site. That is why they go into one commit, with a message that names the whole change.

5. **Push.** Type:

   ```
   git push
   ```


© 2026 559 Solutions LLC

The last line ends in `main -> main`.

6. **Check that Git is done.** Type `git status`. It should say `nothing to commit, working tree clean`. Then type `git log --oneline`. Your newest commit, `Style the site with one shared stylesheet`, is on top, with your Module 1 commits under it.

7. **Check the live page (NEW: checking that the stylesheet arrived).** Wait a few minutes (GitHub Docs: "It can take up to 10 minutes for changes to your site to publish"), then open `https://your-username.github.io/my-site/` and press F5. You should see your colors and the nav bar. If the live page shows black serif text on white, the HTML arrived but the stylesheet did not load. Press F12, open the **Console** tab, and look for a red error that names the stylesheet. Online, a missing file is reported as `404 (Not Found)` instead of `ERR_FILE_NOT_FOUND`. See *Troubleshooting*, problems 1 and 5.

8. **Test the live site.** Run the test from Project step 15 again on the live site: both pages styled, the nav in the same spot on both, all six nav clicks working, and no red errors in the Console.

> **Checkpoint.** `git status` says `nothing to commit, working tree clean`. `git log --oneline` shows your new commit on top. Your live address shows both pages with your colors and the Flexbox nav, and the Console has no red errors.

© 2026 559 Solutions LLC

PART 4

Troubleshooting: five common problems

Each problem has the symptom you see, why it happens, and the fix. After each fix, save, press F5, and run the test from Project step 15 again.

1. A page has no styles at all

Symptom. The page is black serif text on white, with blue underlined links, as if you never wrote any CSS. The other page may still be styled.

Why. The browser could not load the stylesheet. The `href` does not match the file's real name or place. The usual causes are `style.css` instead of `styles.css`, a capital letter (`Styles.css`), a file saved as `styles.css.txt`, or `styles.css` saved in another folder. The page itself shows no message, because the HTML is fine (the Console does show one). Only the second request, for the stylesheet, failed.

Fix. Press F12 and open the **Console** tab. A red `Failed to load resource` error confirms that the stylesheet did not load. Then compare three things letter by letter: the `href` in this page's `<link>`, the file name in the VS Code Explorer, and its place (it must sit in `my-site`, next to `index.html`). Fix the `href` or move the file, save, and refresh. If only the live site is unstyled, look for a capital letter, as in Module 1's troubleshooting: Windows ignores the difference, a web server may not. If File Explorer shows the file as a "Text Document", turn on **View → Show → File name extensions** and rename it so it ends in `.css` only.

2. One rule "does nothing" because another rule wins

Symptom. You wrote a rule, saved and refreshed, and the element did not change. Other rules in the same file work.

Why. Another rule sets the same property on the same element, and it wins. Either it is more specific (a class beats an element, an id beats a class), or it has the same specificity and comes later in the file. In Project step 12, `p { background-color: yellow; }` came last and still lost to `.note` on the note paragraphs. A missing colon or semicolon in the line above can also break a declaration without any message (Concept 2).

Fix. Find every rule that could match the element and sets the same property. Score each selector as in Concept 5 and decide which one wins. Then change the rule that should win, or remove the one that should not. If no other rule matches, read your rule and the line above it for a missing colon, semicolon or brace.

3. A class rule does not apply

Symptom. Your `.note` box shows on one page but not on the other, or not at all.

© 2026 559 Solutions LLC

Why. The class name in the HTML and the name in the CSS differ. CSS needs them to match exactly, including capital letters. Common slips: `class="ntoe"`, `class="Note"`, a dot in the HTML (`class=".note"`), or no dot in the CSS (`note {` instead of `.note {`). The browser gives no error, because a class that matches no rule is allowed.

Fix. Copy the class name from the CSS (without the dot) and compare it with the HTML, letter by letter. The dot belongs only in the CSS selector, the same way `#` belongs only in a link and not in an `id` (Module 1).

4. The nav links stack in a column, or the gap does nothing

Symptom. The nav links sit one under the other, or they stay in a row with only the small spaces of plain text, and your `gap` does nothing.

Why. `display: flex` is on the wrong element. If it is in the `nav a` rule, each link becomes its own flex container, which starts on a new line, so the links stack (Project step 13). If it is missing from the `nav` rule, the `nav` has no flex items, so `gap` has nothing to space out. If it is only on `header`, the `<h1>` and the `<nav>` line up, but the links inside the nav do not get the `gap`.

Fix. Put `display: flex;` in the `nav` rule, the parent of the links, next to `gap`. Remove it from `nav a`. Keep the second `display: flex;` in the `header` rule: that one lines up the site name and the nav.

5. Your change does not show, or the live site shows the old look

Symptom. You edited `styles.css`, but the page looks the same as before. Or your computer shows the new look and the live site does not.

Why. On your computer, the usual cause is that the change was not saved, or the browser tab shows a different copy of the page. A dot on the VS Code tab instead of an ✕ means unsaved changes. Online, GitHub Pages may not have published your push yet, or the browser may still be using a copy of the stylesheet it saved earlier.

Fix. Press Ctrl+S, then F5. Check that the address bar ends in `my-site/index.html` or `my-site/about.html`, not a file in `practice-02`. For the live site, run `git status`: if it lists changed files, finish the save routine. If it is clean, wait a few minutes. Then press **Ctrl+F5**, which reloads the page and its files without the saved copies. Edge and Chrome both have this shortcut.

© 2026 559 Solutions LLC

PART 5

Self-assessment

Rate yourself on each line: 1 Beginning, 2 Developing, 3 Proficient, 4 Ready to advance. Aim for at least **3 (Proficient)** on every line before you start Module 3. If a line is a 1 or a 2, reread that Core Concept and redo its Exercise or Project step.

Skill	1 Beginning	2 Developing	3 Proficient	4 Ready to advance
I can link one stylesheet from several pages and find a wrong href with the Console	☐	☐	☐	☐
I can name the selector, property, value and declaration in a rule, and spot a missing colon or semicolon	☐	☐	☐	☐
I can set color, background, font, line height, margin and padding, and say which of them are inherited	☐	☐	☐	☐
I can choose between an element selector, a class selector and a descendant selector such as nav a	☐	☐	☐	☐
I can write a reusable class with a name that says what the thing is, and use it on more than one page	☐	☐	☐	☐
I can predict which of two rules wins, using specificity first and source order only for a tie	☐	☐	☐	☐
I can tell block boxes from inline boxes and explain why width does nothing on a link	☐	☐	☐	☐
I can lay out a nav bar with display: flex on the parent, and use gap, justify-content and align-items	☐	☐	☐	☐

Three questions to answer in writing

1. In Project step 12, the test rule came last in the file but did not turn the `.note` paragraphs yellow. Explain why, in your own words.
2. Pick one mistake you made in this module. What did the page or the Console show, and how did you find the cause?
3. Your site gets a third page. List everything you must add or check so that it looks like the other two, in order.

© 2026 559 Solutions LLC

PART 6

Answers

Try each question on your own first, in writing. Then compare. The section in brackets is where the lesson or the project explains it.

Check your understanding

1. **No styles at all.** First, the `<link>` line: it must be inside `<head>`, with `rel="stylesheet"`, and its `href` must match the file's name exactly (`styles.css`, not `style.css`). Second, the file itself: is it called exactly that, and is it in the same folder as the page? Press F12 and open the **Console**: a red `Failed to load resource` error confirms that the stylesheet did not load. (Core concept 1; Exercise step 8)
2. **`.note` against a later `p`.** Green. The class selector `.note` is more specific than the element selector `p`, so it wins even though `p` comes later. Source order decides only when specificity ties. (Core concept 5)
3. **`p a { width: 200px; }`.** Nothing changes. The link is an inline box, and MDN says that for an inline box "The `width`, `height`, and top and bottom margins will have no effect." The selector is fine: `p a` does match the link. The box type is the reason. (Core concepts 4 and 6)

Common misconception

"The rule lower down in the stylesheet always wins." Only when the two rules have the same specificity. In the cascade test, line 3 has `class="note"` and `id="special"`. The `.note` rule comes after `#special` in the file, but the line is purple, not green, because an id selector is more specific than a class selector. Order is only the tie-breaker. You saw the same thing in Project step 12: the `p` test rule came last and still lost to `.note`. (Core concept 5; Project step 12)

Exercise checks

- **Step 6.** Your page shows your background color, font and text color, two heading colors, the highlighted paragraph with its own background, and a nav bar with its own background and white links in a row with space between them.
- **Step 7.** With `justify-content: center`, the nav links move to the middle of the bar. With a second `.highlight` rule below the first, the second one's background wins: the two rules have the same specificity, so the later one wins. (Core concepts 5 and 7)
- **Step 8.** With `href="style.css"`, every style is gone, the page shows no message, and the Console shows `Failed to load resource: net::ERR_FILE_NOT_FOUND`. (Core concept 1)
- **Step 9.** Only the declaration with the missing colon stops working; for example, with `color #d1495b;` the heading keeps the default color while its other declarations still work. If you deleted a semicolon instead, that declaration *and the next one* stop working. (Core concept 2)

© 2026 559 Solutions LLC

Self-assessment questions

1. **Why the test rule did not turn the `.note` paragraphs yellow.** When two rules disagree, the more specific one wins. `.note` is a class and `p` is an element, and a class beats an element. Source order decides only a tie, so being last did not help the `p` rule. The `.lead` paragraph did turn yellow, because the `.lead` rule sets only `font-size` and does not disagree about the background. (Project steps 10 and 12)
2. **A mistake you made.** There is no single answer. A good answer names the mistake, quotes what the page or the Console really showed, for example a red `ERR_FILE_NOT_FOUND`, and says how you found the cause, for example by checking which page links which file.
3. **A third page that looks like the other two.** First, type `<link rel="stylesheet" href="styles.css">` as the last line inside its `<head>`, and check the spelling of `styles.css`. Second, copy the same header and footer from your other pages. Third, add a link to the new page in the nav of every page. Fourth, use the classes you already have, like `.note` and `.lead`, where they fit. You do not need a new stylesheet. Fifth, save, press F5, and click between the pages: the nav must sit in the same spot and the header must not jump. Sixth, open the Console and check for red errors. Last, publish and run the test again on the live site. (The brief; Project steps 4, 6, 8, 10, 14, 15 and 16)

Project checks

- **Brief.** `my-site` holds `index.html`, `about.html`, `image-one.jpg` and `styles.css`. Both pages have the same `<link rel="stylesheet" href="styles.css">` as the last line inside `<head>`, and `styles.css` has the four groups: site-wide basics, header and nav, page areas, reusable classes. (The brief)
- **Step 5.** The Home page is styled; the About page is still plain, because only the Home page links the stylesheet so far. (Step 5; Core concept 1)
- **Step 8.** The header is one dark band, with the site name on the left and the nav as one row on the right, in the same place on both pages. (Step 8; Core concept 7)
- **Step 11.** One edit to the header color in `styles.css` changes both pages. (Step 11; Core concept 1)
- **Step 12.** Every paragraph turns yellow except the `.note` paragraphs. The `.lead` paragraph turns yellow too, because `.lead` does not set a background. (Step 12; Core concept 5)
- **Step 13.** With `display: flex` on `nav a`, the links stack in a column and the header grows taller. Each link became a block-level flex container, and the `nav` was no longer a flex container. (Step 13; Core concepts 6 and 7)
- **Step 14.** With `href="style.css"` on the About page only, the About page loses every style while the Home page keeps them, and the Console shows `Failed to load resource: net::ERR_FILE_NOT_FOUND`. The page's link was wrong; `styles.css` was fine. (Step 14; Core concept 1)
- **Finishing work.** `git status` says `nothing to commit, working tree clean`, `git log --oneline` shows `Style the site with one shared stylesheet` on top, and the live address shows both pages styled with no red errors in the Console. (Part 3, steps 6 to 8)

© 2026 559 Solutions LLC

NEXT

Module 3: Responsive Layout.
Free lesson: 559solutions.com/code559/module-03-responsive-layout/

© 2026 559 Solutions LLC

About 559 Solutions

559 isn't just a name. It's where I started: 559 East 37th Street, in the Ida B. Wells Homes on Chicago's South Side. 559 Solutions carries that start into the work: plain language, steady delivery, and solutions people actually use.

The blueprint and the open door

Every 559 Solutions page is drawn like a floor plan: a wall, a window, another wall, and an open door. The plan is how we solve problems, large or small, in any corner of information technology. That covers a nonprofit's tangled spreadsheet and a company's enterprise architecture alike. Every solution moves through the same four steps:

- **Inform:** name the problem, and why it matters.
- **Explain:** show how the solution works, one idea at a time.
- **Illustrate:** draw it, so everyone sees the same plan.
- **Apply:** build it, and put it to work.

In every module, the same four steps sit behind the badges; there the first one is called Introduce. The open door is the point of the plan: a way in, for clients and learners alike.

Code559

Code559 is the teaching side of 559 Solutions. It teaches more than code. It teaches a developer's mindset: predict before you run, read the error, change one thing at a time, test it, save it, and check it live. You work with the real tools developers use every day: VS Code, Git, GitHub, the browser's DevTools and Node. You build working code you can publish, the way a developer does on the job. Think of it as a day in the life of a developer, built into every module.

No jargon. No drama. Just solutions that serve the mission.

— J. R. Wells

559solutions.com

© 2026 559 Solutions LLC